

Test # 11

You've survived the test. The results will arrive soon. 🎉

Your score: 83% 25 / 30

Duration: 0:12:15

1.

What is the primary reason a HashSet does not allow duplicate elements?

1 / 1 point

Your Answer: ✓ Correct

☐ It uses == to compare elements.

☒ It uses hashCode() and equals() to determine equality.

☐ It automatically sorts elements.

☐ It stores elements using indexes.

2.

```
class Parent {

    void show() {

        System.out.println("Parent");

    }

}

class Child extends Parent {

    void show() {

        System.out.println("Child");

    }

}

public class Test {

    public static void main(String[] args) {

        Parent p = new Child();

        p.show();

    }

}
```

1 / 1 point

What will be the output?

Your Answer: ✓ Correct

☐ Parent

☒ Child

☐ Compilation error

☐ Runtime exception

3.

Which statement about method overloading is correct?

1 / 1 point

Your Answer: ✓ Correct

☐ Return type alone can distinguish overloaded methods

☒ Method name and parameter list must differ

☐ Access modifier must differ

☐ Static and non-static methods cannot be overloaded

4.

```
class A {

    private void test() {

        System.out.println("A");

    }

}

class B extends A {

    private void test() {

        System.out.println("B");

    }

}


```

1 / 1 point

What happens here?

Your Answer: ✓ Correct

☐ Compilation error because test() is overridden

☐ B.test() overrides A.test()

☒ Both methods are independent

☐ Runtime exception

5.

Which member cannot be inherited by a subclass?

1 / 1 point

Your Answer: ✓ Correct

☐ Static method

☒ Private method

☐ Public method

☐ Protected method

6.

```
class Test {

    static int x = 10;

    static {

        x += 20;

    }

    public static void main(String[] args) {

        System.out.println(x);

    }

}
```

1 / 1 point

Your Answer: ✓ Correct

☐ 10

☐ Compilation error

☒ 30

☐ 20

7.

Which statement about an abstract class is correct?

1 / 1 point

Your Answer: ✓ Correct

☒ An abstract class can contain abstract and concrete methods

☐ An abstract class cannot have concrete methods

☐ An abstract class cannot have constructors

☐ An abstract class must contain an abstract method

8.

Which statement about an abstract class is correct?

1 / 1 point

Your Answer: ✓ Correct

☒ An abstract class can contain abstract and concrete methods

☐ An abstract class cannot have concrete methods

☐ An abstract class cannot have constructors

☐ An abstract class must contain an abstract method

9.

```
String s1 = new String("Java");

String s2 = "Java";

System.out.println(s1 == s2);
```

1 / 1 point

Your Answer: ✓ Correct

☐ true

☒ false

☐ Compilation error

☐ Runtime Error

10.

```
String a = "Ja";

String b = "va";

String c = a + b;

System.out.println(c == "Java");
```

-0.25 / 1 point

Your Answer: ✗ Incorrect

☒ true

☐ false

☐ Compilation error

☐ Runtime Error

11.

```
final String a = "Ja";

final String b = "va";

String c = a + b;

System.out.println(c == "Java");
```

1 / 1 point

Your Answer: ✓ Correct

☒ true

☐ false

☐ Compilation error

☐ Runtime Error

12.

What happens when HashMap exceeds its threshold?

1 / 1 point

Your Answer: ✓ Correct

☐ It is deleted

☐ It throws an exception

☐ It becomes immutable

☒ It resizes and redistributes entries

13.

If a.equals(b) returns true, what must be true?

1 / 1 point

Your Answer: ✓ Correct

☐ Their hash codes must differ

☒ Their hash codes must be equal

☐ Their references must be identical

☐ They must have the same class

14.

Can two unequal objects have the same hashCode?

1 / 1 point

Your Answer: ✓ Correct

☐ Never

☒ Yes

☐ Only in HashSet

☐ Only in TreeSet

15.

What happens if a HashMap key's fields used by hashCode() are modified after insertion?

-0.25 / 1 point

Your Answer: ✗ Incorrect

☒ HashMap automatically relocates it

☐ Lookup may fail

☐ Compilation error

☐ Key becomes null

16.

Is finally executed if System.exit(0) is called?

1 / 1 point

Your Answer: ✓ Correct

☐ Yes

☒ No

☐ Only with checked exceptions

☐ Depends on JVM

17.

What is the difference between throw and throws?

1 / 1 point

Your Answer: ✓ Correct

☐ They are identical

☐ throw declares exceptions; throws creates exceptions

☒ throw actually throws an exception; throws declares possible exceptions

☐ throws can only be used inside catch

18.

What is the difference between throw and throws?

1 / 1 point

Your Answer: ✓ Correct

☐ They are identical

☐ throw declares exceptions; throws creates exceptions

☒ throw actually throws an exception; throws declares possible exceptions

☐ throws can only be used inside catch

19.

Which method starts a new thread?

1 / 1 point

Your Answer: ✓ Correct

☐ run()

☒ start()

☐ execute()

☐ join()

20.

What happens if you directly call run()?

1 / 1 point

Your Answer: ✓ Correct

☐ New thread starts

☒ Current thread executes run()

☐ Compilation error

☐ JVM creates daemon thread

21.

What does map() do?

1 / 1 point

Your Answer: ✓ Correct

☐ Filters elements

☒ Transforms each element

☐ Sorts elements

☐ Terminates the stream

22.

What does Collectors.groupingBy() produce by default?

1 / 1 point

Your Answer: ✓ Correct

☐ Set

☐ Optional

☐ Queue

☒ Map

23.

Where are objects generally allocated?

1 / 1 point

Your Answer: ✓ Correct

☐ Stack

☒ Heap

☐ Method Area

☐ MetaSpace

24.

StackOverflowError is commonly caused by:

1 / 1 point

Your Answer: ✓ Correct

☐ Excessive object creation

☒ Infinite/deep recursion

☐ Too many database connections

☐ Hash collisions

25.

OutOfMemoryError can commonly occur due to:

1 / 1 point

Your Answer: ✓ Correct

☒ Heap exhaustion

☐ Syntax error

☐ Invalid method overload

☐ Integer overflow

26.

```
Integer a = 100;

Integer b = 100;

System.out.println(a == b);
```

1 / 1 point

Your Answer: ✓ Correct

☒ true

☐ false

☐ Compilation error

☐ Runtime exception

27.

```
public class Test {

    static int x = 10;

    public static void main(String[] args) {

        Test t1 = new Test();

        Test t2 = new Test();

        t1.x = 20;

        System.out.println(t2.x);

    }

}
```

1 / 1 point

Your Answer: ✓ Correct

☐ 10

☒ 20

☐ 0

☐ Compilation error

28.

```
class Employee {

    private int id;

    private String name;

    // getters/setters

    @Override

    public int hashCode() {

        return Objects.hash(id, name);

    }

    @Override

    public boolean equals(Object obj) {

        // compares id and name

    }

}
```

-0.25 / 1 point

You do:

```
Employee emp = new Employee(101, "Rahul");

Map<Employee, String> map = new HashMap<>();

map.put(emp, "Developer");

emp.setName("Amit");

System.out.println(map.get(emp));
```

Your Answer: ✗ Incorrect

☒ It always prints Developer

☐ It prints null because the object's hash code may have changed

☐ Compilation error

☐ HashMap automatically moves the object to the correct bucket

29.

You have a cache:

-0.25 / 1 point

```
ConcurrentHashMap<String, Integer> cache = new
ConcurrentHashMap<>();

Two threads execute:

if (!cache.containsKey("user1")) {

    cache.put("user1", 100);

}
```

The developer says: We are using ConcurrentHashMap, so this code is thread-safe.

Is the developer correct?

Your Answer: ✗ Incorrect

☒ Yes

☐ No, because the combined check-and-put operation isn't atomic

☐ Yes, because ConcurrentHashMap synchronizes every operation

☐ No, because ConcurrentHashMap doesn't support multiple threads

30.

Your application processes millions of requests. A developer writes:

1 / 1 point

```
public class UserCache {

    private static final List<User> USERS = new ArrayList<>();

    public static void add(User user) {

        USERS.add(user);

    }

}
```

Every request calls:

```
UserCache.add(user);
```

After several hours, the application throws:

```
java.lang.OutOfMemoryError: Java heap space
```

The developer says: But Garbage Collection is enabled, so the objects should be removed.

What is the most likely reason?

Your Answer: ✓ Correct

☐ GC doesn't work with ArrayList

☐ Static variables are automatically deleted after each request

☐ ArrayList stores objects in Stack memory

☒ Objects are still strongly reachable through the static list